Project 1 Queue Simulation – Part 1

Objective:

After this experiment, the students should understand:

i. The basic elements of discrete event simulation. In particular, the notion of an event scheduler for simulating discrete event systems and the generation of random variables for a given distribution.

ii. The behavior of a single buffer queue with different parameters C, L, K and λ (as defined below), as it is a key element of a computer network.

iii. The need to be careful about when a system should be observed to obtain the correct performance metrics.

Programming Languages: Students can use C/C++/Python languages.

Overview:

The performance of a communication network is a key aspect of network engineering. Performance measures like delay (how long a packet stays in a system or takes to be delivered), loss ratio (the percentage of the packets that are lost in the system), etc. are some of the criteria that network engineers are trying to predict or measure since they affect the Quality of Service (QoS) offered to the users. In order to do so, models are built to help understand the system and predict its behavior. A *model* of a system is a mathematical abstraction that captures enough of the system's features to give good estimates of the system's performance. The purpose is to build a model and to analyze it to get results on the performance we can expect. Hence depending on the performance we want to study (e.g., reliability, loss, delay,…) we will build a different model of the system. In real situations the complexity of a system can be extremely high (this is the case in a network), and hence the model describing it can be very complicated. In that case it may be difficult to analyze the model exactly. We can either simplify the model (which is an art, since we need to keep it relevant) or try to solve the complicated model

using approximations. In both cases, the results thus obtained SHOULD be validated by *simulation*.

Simulation is not only useful for validating approximate solutions but also in many other scenarios. During the design of a system, it allows us to compare potential solutions at a relatively low cost. It is also very useful to dimension a system, i.e. to decide how much resources to allocate to the system based on an a-priori knowledge of the inputs. It is also used to check how potential modifications of an existing system could affect the overall performance. Simulation is also especially useful when it is difficult or impossible to experiment with the real system or take the measurements physically, e.g., measuring the performance of the Internet as the whole, performing nuclear reactions, airplane crash tests, etc. With the use of computer simulations, the above mentioned scenarios can be reproduced to help us gain insights about the behavior of the system.

To perform a simulation we need a model of the system, as well as models for its input(s). This will be discussed later in more detail.

In communication networks, the most frequent basic-block model we encounter is a queue. You are going to design a simulator, and use it to understand the behavior of two basic types of queues.

Background Materials:

In this experiment, you will be asked to simulate a FIFO (First-In-First-Out) queue (see Figure 1). In general a queue is described by three to five parameters, with a slash ("/") separating each of them, e.g., G/G/1/K/FIFO. This is referred to as Kendall notation. We are interested in the first four parameters (the fifth parameter is the service discipline, which is FIFO by default).

The first and second parameters are descriptions of the arrival process and the service process, respectively. "M" means *memoryless* or *Markovian*, "D" means *Deterministic* and "G" means *General*. The third one is the number of servers, and the fourth one is the size of the buffer. So, for example M/M/c means that both the arrival and service processes are Markovian, and that there are c servers. If the fourth parameter

is not present, it means that the buffer size is infinite. M/G/1 means the queue has one server, the arrival process is Markovian and the service process is general (some non-Markovian distributions, e.g., Gaussian). If the arrival process is M, it means that the distribution of the time between successive arrivals (also called inter-arrival time) is identical for all inter-arrivals, is independent from one interarrival to another and is exponentially distributed. This is equivalent to saying that the process counting the arrivals is Poisson. If the service process is M, it means that the distribution of the service time is identical for each customer/packet, is independent from one customer to another and is exponentially distributed. The exponential distribution is often used in performance evaluation because it is an easy distribution to use (it is characterized by only one parameter) and it is adequate to model call durations and call arrivals in a telephone system. In a data communication system such as the Internet, it is not so adequate for modeling the arrival process of packets but is still used due to its simplicity. If the service process is D, it means that each customer/packet will receive the same constant service time.

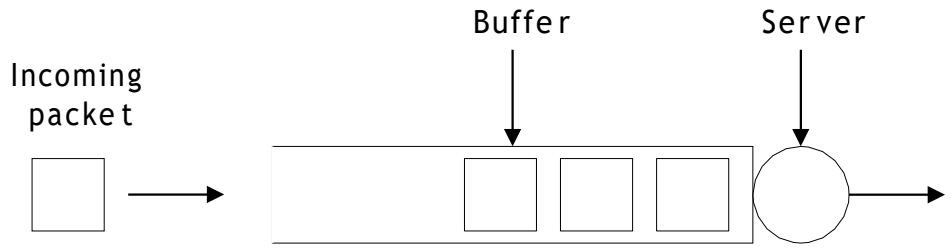


Figure 1: Model of a queueing system

The queues you need to simulate in this experiment are M/M/1.

Simulation Basics

A simulation model consists of three elements: *Input variables, Simulator and Output variables*. The simulator is a mathematical/logical relationship between the input and the output. A simulation experiment is simply a generation of several instances of input variables, according to their distributions, and of the corresponding output variables from the simulator. We can then use the output variables (usually some statistics) to

estimate the performance measures of the system. The generation of the input variables and the design of the simulator will be discussed below.

Generation of input variables with different distributions:

You will need a uniformly distributed random variable (r.v.) generator. In particular, you will need to generate U(0,1), a uniform r.v. in the range (0,1). It is important to use a "good" uniform r.v. generator for better simulation results. A good uniform r.v. generator will give you independent, identically distributed (i.i.d.) uniform random variables. You can test how good a uniform r.v. generator is by checking the mean (0.5) and variance (1/12) of the random samples you have generated. The mean and variance of a set of such samples should be very close to the theoretical values for 1000 or more samples. Note that, in general, it is not enough to check the mean and variance to conclude on the validity of your random generator, but we will assume that it is a good enough test for our purposes. Since this is not a course on simulation we cannot spend too much time on the topic of the generation of a r.v. but note that you should always start by verifying the performance of your r.v. generator by checking that you are indeed generating what you want.

If you need to generate a r.v. X with a distribution function[1] F(x), all you have to do is the following:

- Generate U~U(0,1)
- Return $X = F^{-1}(U)$

where $F^{-1}(U)$ is the inverse of the distribution function of the desired random variable.

Generating exponential random variables

Assume that we want to generate an exponential random variable x with average $1/\lambda$, where λ is the rate parameter. We will use the inverse method. We start by the exponential **cumulative distribution function (CDF)** given as:

$$F(x) = P(X \leq x) = 1 - e^{-\lambda x} \quad (1)$$

Set $F(x)$ equal to U which is a uniform random variable

[1] To learn more about density and distribution functions, please refer to the book by Alberto Leon-Garcia, "Probability and Random Processes for Electrical Engineering", second edition (July 1993), pp. 87.

$$F(x) = U \tag{2}$$

Then, substitute from (1) in (2) and solve for x

$$1 - e^{-\lambda x} = U$$

Then, we have

$$x = -\left(\frac{1}{\lambda}\right) * \ln(1 - U)$$

where U is a uniformly generated number and x is the exponential random variable

<u>Question 1</u> (50 points):

How to generate an exponential random variable with parameter λ from U (0,1)? Show all your calculations. Write a short piece of C/C++/Python code to generate 1000 exponential random variable samples from your equation, using λ=75. What is the mean and variance of the 1000 random variables you generated? Do they agree with the expected value and the variance of an exponential random variable with λ=75**?** (Compare them with the theoretical values of mean and variance for exponential random variables).

Please write down all your analytical derivations/calculations, the snapshot of your codes with annotations/comments, and the snapshot of your coding running results. Give explanations about why your results are correct or reasonable?

(Hint: function **rand ()** returns a pseudo-random number in the range of 0 to RAND_MAX uniformly; RAND_MAX: is a constant whose default value may vary between implementations, but it is granted to be at least 32,767.)

Important note: **rand ()** returns a random positive integer in the range from 0 to 32,767. This function can be thought of as rolling a die with 32,767 faces. The numbers are generated by a mathematical algorithm which when given a starting number **(called the "seed")**, always generates the same sequence of numbers. Since the same sequence is generated each time the seed remains the same, the rand() function generates a pseudo-random sequence. To prevent the same sequence from being generated each time, use **srand((unsigned)time(0)) before using rand ()** to change the seed value.

Install C/C++ Programming IDE (e.g., Code ::Blocks)

If you face the following environment error: **"Can't find file executable in your configured search path for GNC/GCC compiler"**

Go into the Settings menu, then select compiler in the dropdown, and select the Toolchain executables tab. Make sure the Compiler's installation directory textbox matches the folder you installed GCC into. For me, this is **C:\Program Files (x86)\CodeBlocks\MinGW,** you can click Auto-detect, to locate your installed GCC/GNC compiler path.

```
#include<iostream> // header input and output stream for printing in C++
#include<malloc.h> // header for memory allocation function
#include<time.h> // header for time functions
#include<stdlib.h> // This header defines several general purpose functions
#include<math.h> // THis header supports arithmetic functions

using namespace std;
```

Project 1 Queue Simulation – Part 2

Objective:

After this experiment, the students should understand:

iv. The basic elements of discrete event simulation. In particular, the notion of an event scheduler for simulating discrete event systems and the generation of random variables for a given distribution.

v. The behavior of a single buffer queue with different parameters C, L, K and λ (as defined below), as it is a key element of a computer network.

vi. The need to be careful about when a system should be observed to obtain the correct performance metrics.

Programming Languages: Students can use C/C++/Python programming languages

1. Designing a Discrete Event Simulator (DES)

Discrete Event Simulation (DES) is commonly used to study the performance of evolving systems whose states change at discrete points in time in response to external and internal events. A DES simulator consists of a set of time-ordered events E_i, $i = 1,\ldots, N$, along with their occurrences time, such that $t(Ei) < t(Ei+1)$ for all i as shown in Table 1. The time of the last event , which is $t(Ei+3)$ in this case, is the called the simulation time. Note that **there is a difference between the simulation time and the execution time of your code**. The execution time is how long your code will run for in order to simulate a system for an amount of time equal to the simulation time.

Table 1. *Discrete Event Simulation (DES)*

Event	Time
E_i	$t(E_i)$
E_{i+1}	$t(E_{i+1})$
E_{i+2}	$t(E_{i+2})$
E_{i+3}	$t(E_{i+3})$

To perform DES, you need to create events and their corresponding occurrence time. But when should the DES terminate? By performing DES, we are performing a statistical experiment. In order to get good results in a statistical experiment, your simulation time should be large enough to truly represent the system under consideration, which is a queue in this case. An easy way to get stable simulation results is to run the experiment for T seconds, take the result, run the experiment again for 2T seconds and see if the expected values of the output variables are similar to the output from the previous run. For example, the difference should be within 5% of the previous run. If the results agree, you can claim the result is stable. If not, you try again for 3T, 4T, … If you cannot seem to find a proper T, it may mean that your system is unstable. For the purpose of this project, the value of T should be starting from more than 1000 seconds but you should still use the procedure described above to check the stability (and hence the validity) of your results.

2. Simulating a Queue

Systems involving queues are generally simulated with a DES simulator. A queue comprises two components: a buffer and one or many servers.

1. In order to simulate a queue using DES, you need to select the simulation time T and create queue events and their corresponding time. In a queue, you have three types of events:
   a. **Packet arrival:** Based on the type of the arrival process specified in the Kendall notation, generate a random variable that represents the arrival times of packets to a queue. For example, in M/M/1 queue, the packet arrival will follow an exponential distribution. Use the inverse method previously explained to generate packet arrivals for a period of time equal to the simulation time T.

   b. **Packet departure:** Calculate the departure time of a packet based on the state of the queue, i.e., whether the queue has packets or it is idle. If the queue has packets, then the departure time of a packet pkt_i will be the departure time of the previous packet pkt_{i-1} plus the transmission time of pkt_i . If the queue is idle, then the departure time of packet pkt_i will be its arrival time plus its transmission time. See Table 2.

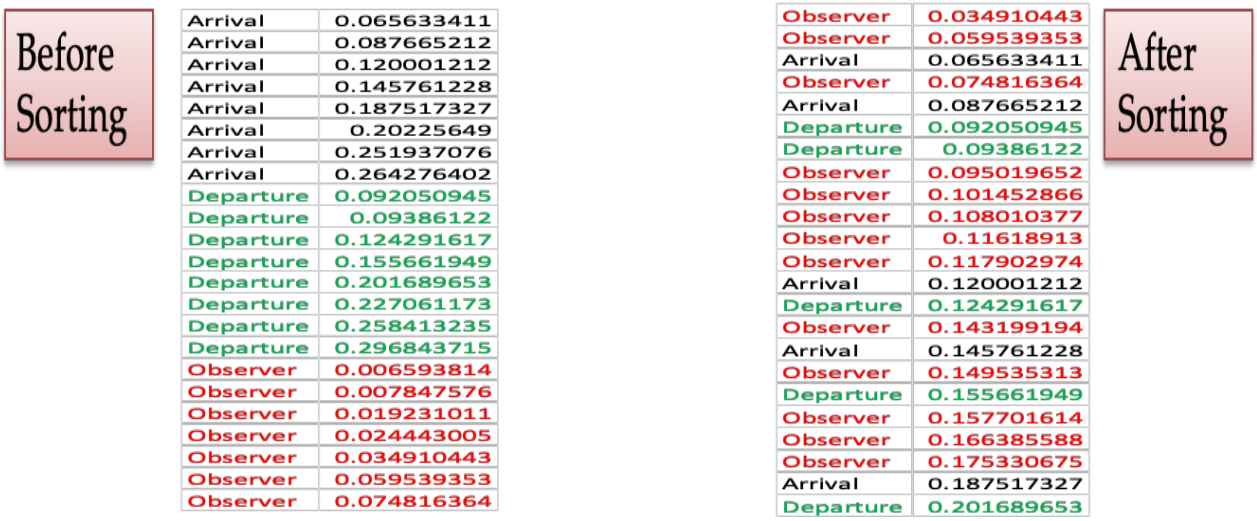
Table 2. Example of generating arrivals and departures

Event	Arrival Time	Length (bits)	Service Time	Departure Time
Arrival	0.01172	3694.4	0.0037	0.0154136
Arrival	0.01942	13438	0.0134	0.032857
Arrival	0.04083	7341.3	0.0073	0.0481729
Arrival	0.06028	24220	0.0242	0.0845039
Arrival	0.0734	9477.1	0.0095	0.0939809
Arrival	0.08096	6789.5	0.0068	0.1007704
Arrival	0.09245	6895.9	0.0069	0.1076663
Arrival	0.11049	4626.9	0.0046	0.1151162

   c. **Observer:** In order to monitor the queue, you need to generate observer events at which you are going to record the state of the queue. **Generate a set of random observation times according to the packet arrival distribution with rate at least 3 times the rate of the packet arrival.** At an observer event, you will record the state of the queue which can be done by recording the following: the time-average of the

number of packets in the entire queue system, E[N], the proportion of time the server is idle (i.e., the system is empty) P_{IDLE} and in the case of a finite queue, the probability P_{LOSS} that a packet will be dropped (due to the buffer being full when it arrives). The measurement and recording of the state should be performed such that we have statistically representative information on the performance that we are interested in. In order to record the state of the queue, you need to have three counters: N_a = number of packet arrivals, N_d = number of packets departures, and N_o = number of observations. The onset is on you to find out how to use the previous three counters to calculate the state of the queue system, i.e., E[N], P_{IDLE}, and P_{LOSS} (in case of finite queue).

2. After generating all the three events, put all the events in a list and sort them



according to time as shown in the Fig. 2.

Figure 2. DES example

3. **Start processing the events in order from DES**. Based on the event type, you should update the three counters N_a, N_d and, N_o. If the event is observer, calculate the performance metrics of interest, e.g., P_{IDLE}, P_{LOSS}. After an event has been processed, it should be deleted from DES.

2.1 Notations

- λ = Average number of packets generated /arrived (packets per second)
- L = Average length of a packet in bits.
- α = Average number of observer events per second
- C = The transmission rate of the output link in bits per second.
- ρ = Utilization of the queue (= input rate/service rate = Lλ/C)
- E[N] = Average number of packets in the entire queueing system
- P_{IDLE} = The proportion of time the server is idle, i.e., no packets in the queue nor a packet is being transmitted.
- P_{LOSS} = The packet loss probability (for M/M/1/K queue). It is the ratio of the total number of packets lost due to buffer full condition to the total number

of generated packets.

Read all the questions before you start designing your simulator.

M/M/1 Queue: Recall that it is a queue with an infinite buffer.

Question 2:
***1)* (30 points)** Build your simulator for this queue and explain in words what you have done. Show a snapshot of your codes. Define your variables and explain how you compute the performance metrics.

***2)* (20 points)** The packet length will follow an exponential distribution with an average of L = 12000 bits. Assume that C = 1Mbps. Use your simulator to obtain the following relations and draw the corresponding graphs. *Provide comments on all your figures*.

1. E[N], the average number of packets in the queue system as a function of ρ (for 0.25 <= ρ <= 0.95, step size 0.1). **Explain how you do that and compare with the theoretical results.**
2. P_{IDLE}, the proportion of time the system is idle as a function of ρ (for 0.25 <= ρ <= 0.95, step size 0.1). **Explain how you do that and compare with the theoretical results.**